Linear Regression Notes — Hinglish

Nov 7, 2026 · @Razeen

Atlas Skilltech University — ML Course Notes (Hinglish)

1. Introduction to Linear Regression

Linear Regression Kya Hai?

Linear Regression ek **supervised learning algorithm** hai jo **continuous values** predict karta hai. Yeh ek straight line (ya hyperplane) fit karta hai data ke through.

Simple analogy: Agar tum pichle 5 saalon ka data dekho ki experience badhne se salary kitni badi, aur phir predict karo ki 8 saal experience wale ko kitna milega — yahi Linear Regression karta hai.

Types of Linear Regression

Type	Features	Example
Simple Linear Regression	1 input feature	Experience se Salary predict karna
Multiple Linear Regression	2+ input features	Size, Rooms, Age se House Price predict karna
Polynomial Regression	Non-linear patterns	Curved data fit karna

Mathematical Form

Simple:

```
y = B0 + B1*x + error
```

Multiple:

```
y = B0 + B1*x1 + B2*x2 + ... + Bn*xn + error
```

- **y** = target variable (jo predict karna hai)

- **B0** = intercept (y-axis intercept)
- **B1...Bn** = coefficients (har feature ka weight)
- **error (epsilon)** = noise jo explain nahi ho sakta

Linear Regression Assumptions (LINE Rule)

- **L**inearity: X aur Y ka linear relationship hona chahiye
- **I**ndependence: Observations ek doosre se independent hon
- **N**ormality: Residuals normally distributed hon
- **E**qual Variance (Homoscedasticity): Error ka variance constant ho

Linear Regression vs Classification

Aspect	Linear Regression	Logistic Regression
Output	Continuous (1,2,3...)	Binary (0 or 1)
Examples	Price, Score, Age	Spam/Not Spam
Loss Function	MSE	Log Loss
Output Range	-inf to +inf	0 to 1 (probability)

2. Ordinary Least Squares (OLS)

OLS Kya Karta Hai?

OLS woh line dhundta hai jo **sum of squared residuals (SSR)** ko minimize kare.

Rubber band analogy: Socho data points ko vertical rubber bands se line se connect kiya gaya hai. OLS woh line dhundta hai jahan rubber bands ki total khichaav sabse kam ho.

OLS Formulas

Simple Linear Regression ke liye:

```
\beta_1 = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}
(x_i - \bar{x})^2}
```

```
\beta_0 = \bar{y} - \beta_1 \bar{x}
```

Matrix Form (Multiple Regression):

```
\hat{\beta} = (X^T X)^{-1} X^T y
```

Step-by-Step OLS (Scratch)

```
# Manual OLS calculation
x_mean = np.mean(X)
y_mean = np.mean(y)

numerator = np.sum((X - x_mean) * (y - y_mean))
denominator = np.sum((X - x_mean) ** 2)

beta1 = numerator / denominator   # slope
beta0 = y_mean - beta1 * x_mean   # intercept
```

OLS Properties

OLS **BLUE** hai (Best Linear Unbiased Estimator) jab LINE assumptions poori hon:

- **B**est = Minimum variance among all linear estimators
- **L**inear = Parameters ka linear function
- **U**nbiased = On average correct answer deta hai
- **E**stimator = Data se calculate kiya gaya

Goodness of Fit

Metric	Formula	Matlab
SS_total	sum((y - y_mean)^2)	Total variance in y
SS_res	sum((y - y_pred)^2)	Unexplained variance
SS_reg	SS_total - SS_res	Explained variance
R-squared	1 - SS_res/SS_total	Proportion of variance explained

Python Implementation

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

print(f'Intercept: {model.intercept_}')
print(f'Coefficients: {model.coef_}')
print(f'R-squared: {model.score(X_test, y_test)}')
```

3. Cost Functions

Cost Function Kya Hai?

Cost function ek **scoring system** hai jo batata hai model kitna galat hai. Hum cost function **minimize** karte hain taaki best model mile.

Analogy: Jaise ek coach player ki performance score karta hai — jitna score kam, utna kharab performance. Model bhi yahi karta hai — jitna cost kam, utna better model.

Main Cost Functions

Mean Squared Error (MSE)

```
\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2
```

- Large errors ko zyada penalize karta hai (squaring ki wajah se)
- Outliers se highly sensitive
- Units: target variable ki square units (e.g., Rs^2)

Mean Absolute Error (MAE)

```
\text{MAE} = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i|
```

- Outliers se robust (squaring nahi hota)
- Target ke same units mein hota hai
- Gradient always constant (not smooth at 0)

Root Mean Squared Error (RMSE)

```
\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2}
```

- MSE ki same units mein return karta hai target ke
- Most popular metric for reporting

R-Squared (Coefficient of Determination)

```
R^2 = 1 - \frac{\text{SS}_{\text{res}}}{\text{SS}_{\text{tot}}} = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}
```

- 1.0 = Perfect prediction
- 0.0 = Model sirf mean predict kar raha hai
- Negative = Model mean se bhi bura hai!
- 0.85 = 85% variance explain ho raha hai

Comparison Table

Metric	Outlier Sensitive?	Units	Range	Use When
MSE	Yes (squares)	y^2	0 to inf	Optimization (smooth gradient)
MAE	No	y	0 to inf	Outliers present hain
RMSE	Yes	y	0 to inf	Reporting (interpretable)
R^2	No	Unitless	-inf to 1	Model comparison

Python Implementation

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import numpy as np

y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f'MSE:  {mse:.2f}')
print(f'MAE:  {mae:.2f}')
print(f'RMSE: {rmse:.2f}')
print(f'R-squared: {r2:.4f}')
```

4. Gradient Descent

Gradient Descent Kya Hai?

Gradient Descent ek **iterative optimization algorithm** hai jo cost function minimize karne ke liye parameters dhundta hai.

Mountain analogy: Imagine tum ek dense fog mein ek pahar par ho aur neeche utarna chahte ho. Tum sirf apne pair ke neeche ki slope feel kar sakte ho. Gradient Descent bar bar poochta hai: "Mera current position pe slope kahan neeche hai?" aur uss direction mein ek chhota step leta hai — jab tak minimum mil jaye.

Update Rules

```
\beta_j := \beta_j - \alpha \frac{\partial J}{\partial \beta_j}
```

Linear Regression ke liye partial derivatives:

```
\frac{\partial J}{\partial \beta_0} = -\frac{2}{n} \sum_{i=1}^{n}(y_i - \hat{y}_i)
```

```
\frac{\partial J}{\partial \beta_1} = -\frac{2}{n} \sum_{i=1}^{n} x_i(y_i - \hat{y}_i)
```

Learning Rate (Alpha) ka Effect

Alpha Value	Kya Hota Hai?
Too Large	Overshoot — minimum miss karo, diverge hoga
Too Small	Very slow convergence — bahut epochs
Just Right	Smooth convergence

Practical tip: Alpha = 0.01 ya 0.1 se start karo. Loss curve plot karo — agar oscillate kare toh alpha kam karo.

Types of Gradient Descent

Type	Kya Use Karta Hai?	Speed	Accuracy
Batch GD	Poora dataset ek iteration mein	Slow	Stable
Stochastic GD (SGD)	1 sample at a time	Fast	Noisy
Mini-Batch GD	Small batch (32-256 samples)	Balanced	Balanced

sklearn `LinearRegression` OLS use karta hai (closed-form), GD nahi.
GD use hota hai `SGDRegressor` aur Neural Networks mein.

Python: Gradient Descent from Scratch

```
import numpy as np

# Normalize data before GD
X_norm = (X - X.mean()) / X.std()
y_norm = (y - y.mean()) / y.std()
n = len(X_norm)

beta0, beta1 = 0.0, 0.0
alpha = 0.01
epochs = 1000
loss_history = []

for epoch in range(epochs):
    y_pred = beta0 + beta1 * X_norm
```

```
    loss = np.mean((y_norm - y_pred) ** 2)
    loss_history.append(loss)

    # Gradients
    d_b0 = -(2/n) * np.sum(y_norm - y_pred)
    d_b1 = -(2/n) * np.sum(X_norm * (y_norm - y_pred))

    # Update
    beta0 -= alpha * d_b0
    beta1 -= alpha * d_b1

print(f'Final Loss: {loss_history[-1]:.6f}')
```

OLS vs Gradient Descent

Aspect	OLS (Closed Form)	Gradient Descent
Solution	Direct, one-shot	Iterative
Speed (small data)	Very fast	Slower
Speed (large data)	Slow (matrix inversion)	**Faster**
Memory	Needs full matrix	Batch possible
When to use	n < 100K features	Very large datasets, Neural Nets

5. Sklearn Implementation + Residual Plots

Complete Pipeline

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

# Step 1: Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Step 2: Scale (optional for LR, required for regularization)
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)  # fit + transform
X_test_sc = scaler.transform(X_test)        # only transform!

# Step 3: Train
model = LinearRegression()
model.fit(X_train_sc, y_train)

# Step 4: Predict
y_pred = model.predict(X_test_sc)

# Step 5: Evaluate
print(f'R-squared: {r2_score(y_test, y_pred):.4f}')
print(f'RMSE: {np.sqrt(mean_squared_error(y_test, y_pred)):.2f}')
print(f'Intercept: {model.intercept_}')
print(f'Coefficients: {model.coef_}')
```

Model Deployment — Pipeline ke Saath

```
from sklearn.pipeline import Pipeline
import joblib

# Pipeline banao: scaling + model ek sath
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LinearRegression())
])
pipeline.fit(X_train, y_train)  # raw data pass karo!
y_pred = pipeline.predict(X_test)  # raw data pass karo!

# Save karo
joblib.dump(pipeline, 'my_model.pkl')

# Load aur use karo
loaded = joblib.load('my_model.pkl')
prediction = loaded.predict(new_data)  # no manual scaling!
```

Residual Plots

Residual = Actual - Predicted

Acha model ka residuals: random, mean = 0, normally distributed.

Method 1: Manual (Classic Seaborn)

```
import matplotlib.pyplot as plt
import seaborn as sns

residuals = y_test - y_pred

# Residuals vs Predicted
plt.scatter(y_pred, residuals, alpha=0.5)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('Predicted Values')
plt.ylabel('Residuals')
plt.title('Residuals vs Predicted (Method 1)')
plt.show()

# Seaborn residplot
sns.residplot(x=y_pred, y=y_test,
              lowess=True,
              line_kws={'color': 'red'})
plt.title('Residual Plot with LOWESS Trend')
plt.show()
```

Method 2: Modern (PredictionErrorDisplay)

```
from sklearn.metrics import PredictionErrorDisplay

# Actual vs Predicted
PredictionErrorDisplay.from_predictions(
    y_test, y_pred,
    kind='actual_vs_predicted'
)
plt.title('Actual vs Predicted (Method 2)')
plt.show()

# Residuals vs Predicted
PredictionErrorDisplay.from_predictions(
    y_test, y_pred,
    kind='residual_vs_predicted'
)
plt.title('Residuals vs Predicted (Method 2)')
plt.show()
```

Residual Pattern Diagnosis

Pattern	Matlab	Fix
Random scatter	Assumptions poori hain	Model theek hai
Funnel (increasing spread)	Heteroscedasticity	Log transform ya weighted regression
Curve	Non-linearity	Polynomial features add karo
Clusters	Missing categorical feature	Feature engineering karo

6. Bias-Variance Tradeoff + Regularization

Bias vs Variance Kya Hai?

Archery analogy:

- **High Bias:** Consistently ek jagah miss karo (systematic error) — Underfitting

- **High Variance:** Scatter everywhere — unpredictable (Overfitting)
- **Sweet Spot:** Low Bias + Low Variance = Perfect Model

Mathematically:

```
\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}
```

Situation	Train Error	Test Error	Problem
Underfitting	High	High	High Bias (model too simple)
Overfitting	Low	High	High Variance (model too complex)
Good Fit	Low	Low	Balanced

Regularization — Overfitting Rokne Ka Tarika

Regularization cost function mein ek **penalty term** add karta hai large coefficients par.

Ridge Regression (L2)

```
J_{\text{Ridge}} = \text{MSE} + \alpha \sum_{j=1}^{p} \beta_j^2
```

- Coefficients ko shrink karta hai (lekin kabhi exact zero nahi hote)
- Multicollinearity ke liye best
- `alpha=0` = Regular Linear Regression

Lasso Regression (L1)

```
J_{\text{Lasso}} = \text{MSE} + \alpha \sum_{j=1}^{p} |\beta_j|
```

- Coefficients ko EXACTLY zero kar sakta hai (feature selection!)
- Sparse model banata hai
- Best when many features irrelevant hain

ElasticNet (L1 + L2)

```
J_{\text{EN}} = \text{MSE} + \alpha_1 \sum |\beta_j| + \alpha_2 \sum \beta_j^2
```

- Ridge + Lasso dono ka combination
- `l1_ratio` parameter: 0 = Ridge, 1 = Lasso

Alpha Parameter

- Alpha = 0 → Regular Linear Regression
- Alpha very large → All coefficients → 0 (complete underfitting)
- Best alpha: GridSearchCV se dhundho!

Python Code

```
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.model_selection import GridSearchCV

# Ridge
ridge = Ridge(alpha=1.0)
ridge.fit(X_train_scaled, y_train)

# Lasso
lasso = Lasso(alpha=0.1)
lasso.fit(X_train_scaled, y_train)

# ElasticNet
en = ElasticNet(alpha=0.1, l1_ratio=0.5)
en.fit(X_train_scaled, y_train)

# GridSearchCV for best alpha
param_grid = {'alpha': [0.001, 0.01, 0.1, 1, 10, 100]}
grid = GridSearchCV(Ridge(), param_grid, cv=5, scoring='r2')
grid.fit(X_train_scaled, y_train)
print(f'Best alpha: {grid.best_params_}')
```

Quick Decision Guide

Situation	Algorithm
Overfitting, correlated features	Ridge
Many irrelevant features	Lasso (feature selection)
Both above	ElasticNet

Situation	Algorithm
Need to know if regularization helps	GridSearchCV

Section 7: Feature Scaling, Cross Validation aur Model Selection

7.1 Feature Scaling Kyun Zaroori Hai?

Maan lo tumhare paas ek dataset hai:

- Experience: 1 to 10 (years)
- Salary: 30,000 to 150,000 (rupees)

Agar tum directly Linear Regression lagao, toh salary feature experience ko **dominate** kar dega sirf isliye kyunki uski values badi hain. Is problem ko solve karne ke liye **Feature Scaling** karte hain.

3 Main Scaling Methods:

Method	Formula	Range	Kab Use Karein
StandardScaler	(x - mean) / std	-3 to +3 (approx)	Jab data bell-curve jaisa ho
MinMaxScaler	(x - min) / (max - min)	0 to 1	Jab exact range chahiye
RobustScaler	(x - median) / IQR	Varies	Jab outliers zyada hon

---

7.2 StandardScaler — Z-Score Normalization

```
z = \frac{x - \mu}{\sigma}
```

Yahan: mu = mean of the feature, sigma = standard deviation of the feature.

Result: Mean becomes 0, Standard Deviation becomes 1.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)  # fit + transform on training
data
X_test_scaled = scaler.transform(X_test)         # ONLY transform on test
data (never fit!)
```

Important Rule: Test data pe **sirf transform** karo, fit nahi! Warna **data leakage** ho jaayegi.

7.3 MinMaxScaler — Range [0, 1]

```
x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}
```

```
from sklearn.preprocessing import MinMaxScaler
mm_scaler = MinMaxScaler()
X_scaled = mm_scaler.fit_transform(X)
# Ab saari values 0 aur 1 ke beech aayengi
```

7.4 RobustScaler — Outliers ke Liye

```
x_{scaled} = \frac{x - Q_2}{Q_3 - Q_1}
```

Q1, Q2 (median), Q3 quartiles hain. IQR = Q3 - Q1.

7.5 Kab Scaling Karni Chahiye?

Scaling ZAROOR karo:

- Gradient Descent use kar rahe ho
- Ridge, Lasso, ElasticNet regularization
- KNN, SVM, Neural Networks

Scaling ki ZAROORAT NAHI:

- Simple tree-based models (Decision Tree, Random Forest)
- Jab features already same scale pe hon

7.6 Cross Validation — Model ko Properly Evaluate Karo

Problem with Simple Train-Test Split: Ek hi train-test split se model ka evaluation **unstable** ho sakta hai.

Solution: K-Fold Cross Validation

Data ko K equal parts (folds) mein divide karo. Phir K baar train karo:

- Ek fold = test set
- Baaki K-1 folds = training set

Final score = average of all K scores.

```
from sklearn.model_selection import cross_val_score, KFold
from sklearn.linear_model import LinearRegression

model = LinearRegression()
kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=kf, scoring='r2')

print(f"5 Folds ke R Scores: {scores.round(3)}")
print(f"Mean R: {scores.mean():.3f}")
print(f"Std Deviation: {scores.std():.3f}")
```

7.7 Pipeline — Scaling + Model Ek Saath

Best Practice: Scaler aur model ko **Pipeline** mein wrap karo. Isse koi bhi data leakage nahi hogi.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge

# Pipeline banao
pipe = Pipeline([
    ('scaler', StandardScaler()),   # Step 1: Scale features
    ('model', Ridge(alpha=1.0))     # Step 2: Train Ridge model
])

# Cross validation on pipeline
cv_scores = cross_val_score(pipe, X, y, cv=5, scoring='r2')
print(f"Pipeline CV R: {cv_scores.mean():.3f} +/- {cv_scores.std():.3f}")
```

7.8 Model Selection — Kaun Sa Model Best Hai?

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet

models = {
    'Linear Regression': LinearRegression(),
    'Ridge (alpha=1)':   Ridge(alpha=1.0),
    'Lasso (alpha=0.1)': Lasso(alpha=0.1),
    'ElasticNet':        ElasticNet(alpha=0.1, l1_ratio=0.5)
}

results = {}
for name, model in models.items():
    pipe = Pipeline([('scaler', StandardScaler()), ('model', model)])
    scores = cross_val_score(pipe, X, y, cv=5, scoring='r2')
    results[name] = {'mean': scores.mean(), 'std': scores.std()}
    print(f"{name}: {scores.mean():.3f} +/- {scores.std():.3f}")
```

7.9 GridSearchCV — Best Hyperparameter Dhundho

```
from sklearn.model_selection import GridSearchCV

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('model', Ridge())
])

param_grid = {
    'model__alpha': [0.001, 0.01, 0.1, 1.0, 10.0, 100.0]
}

grid_search = GridSearchCV(pipe, param_grid, cv=5, scoring='r2', n_jobs=-1)
grid_search.fit(X_train, y_train)

print(f"Best Alpha: {grid_search.best_params_}")
print(f"Best CV R: {grid_search.best_score_:.3f}")
```

7.10 Summary Table

Concept	Key Point	Code
StandardScaler	Mean=0, Std=1	`StandardScaler()`
MinMaxScaler	Range [0,1]	`MinMaxScaler()`
RobustScaler	Outlier-robust	`RobustScaler()`
K-Fold CV	K baar evaluate	`KFold(n_splits=5)`
cross_val_score	Direct CV scores	`cross_val_score(model, X, y, cv=5)`
Pipeline	Leakage prevent	`Pipeline([...])`
GridSearchCV	Best params	`GridSearchCV(pipe, param_grid)`

Golden Rule: Hamesha Pipeline use karo jab scaling + model saath mein use karo. Kabhi bhi test data pe scaler fit mat karo!

Section 8: Real World Applications, Cheat Sheet aur Capstone

8.1 Linear Regression Real Mein Kahan Use Hota Hai?

Industry	Problem	Features	Target
Real Estate	House Price Prediction	Size, Rooms, Location	Price
HR/Recruitment	Salary Estimation	Experience, Skills, City	Salary
Healthcare	Patient Stay Duration	Age, Diagnosis, Tests	Days
Finance	Credit Score Impact	Income, Debt, History	Score
Manufacturing	Quality Control	Temperature, Pressure	Defects
Marketing	Ad Revenue	Budget, Platform, Season	Revenue

8.2 End-to-End Example — House Price Prediction

Ek real project ka flow:

Step 1: Data Collect karo

```
import pandas as pd
df = pd.read_csv('houses.csv')  # Size, Rooms, Age, Location, Price
print(df.head())
print(df.describe())
```

Step 2: EDA (Exploratory Data Analysis)

```
import seaborn as sns
import matplotlib.pyplot as plt

# Correlation heatmap
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title('Feature Correlations')
plt.show()

# Target distribution
sns.histplot(df['Price'], bins=30, kde=True)
plt.title('House Price Distribution')
plt.show()
```

Step 3: Preprocessing + Pipeline

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV

X = df.drop('Price', axis=1)
y = df['Price']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('model', Ridge())
])

param_grid = {'model__alpha': [0.01, 0.1, 1, 10, 100]}
grid = GridSearchCV(pipeline, param_grid, cv=5, scoring='r2')
```

```
grid.fit(X_train, y_train)

print(f'Best Alpha: {grid.best_params_}')
print(f'Test R2: {grid.score(X_test, y_test):.4f}')
```

Step 4: Model Deploy + Save

```
import joblib

# Save karo
joblib.dump(grid.best_estimator_, 'house_price_model.pkl')

# Load aur predict karo
model = joblib.load('house_price_model.pkl')
new_house = [[1500, 3, 10, 'Mumbai']]  # Size, Rooms, Age, City
predicted_price = model.predict(new_house)
print(f'Predicted Price: Rs. {predicted_price[0]:,.0f}')
```

8.3 Coefficient Interpretation — Real Meaning

```
import pandas as pd

# Standardized coefficients
coef_df = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.named_steps['model'].coef_
}).sort_values('Coefficient', key=abs, ascending=False)

print(coef_df)
```

Interpretation rules:

- **Positive coefficient** (+500): Feature badhne se target badhta hai
- **Negative coefficient** (-300): Feature badhne se target ghatta hai
- **Larger absolute value** = Zyada important feature
- **Standardized coefficients** directly compare kar sakte ho feature importance ke liye

---

8.4 Capstone Project — California Housing

Complete capstone project ka summary:

Dataset: California Housing (20,640 samples, 8 features)

Features:

- MedInc (Median Income)
- HouseAge
- AveRooms, AveBedrms, AveOccup
- Population
- Latitude, Longitude (geographic!)

Target: MedHouseVal (Median House Value in $100,000s)

```
from sklearn.datasets import fetch_california_housing
import pandas as pd

data = fetch_california_housing(as_frame=True)
df = data.frame

print(f"Shape: {df.shape}")
print(df.describe())
```

Key findings from California Housing:

1. MedInc (income) ka sabse strong positive relationship hai price se
2. Geographic features (Lat/Long) important hain — coastal areas expensive
3. High population wale areas cheaper (overcrowding effect)
4. Price $5 pe capped hai (data artifact)

8.5 Complete Cheat Sheet — Linear Regression

Algorithms Quick Reference:

Model	Use Case	sklearn Class	Key Params
Linear Regression	Baseline, interpretable	`LinearRegression()`	None
Ridge (L2)	Multicollinearity	`Ridge(alpha=1)`	alpha
Lasso (L1)	Feature selection	`Lasso(alpha=0.1)`	alpha

Model	Use Case	sklearn Class	Key Params
ElasticNet	Both above	`ElasticNet(alpha, l1_ratio)`	alpha, l1_ratio
SGDRegressor	Very large data	`SGDRegressor()`	learning_rate

Metrics Quick Reference:

Metric	Function	Range	Good =
MSE	`mean_squared_error`	0 to inf	Low
MAE	`mean_absolute_error`	0 to inf	Low
RMSE	`sqrt(MSE)`	0 to inf	Low
R2	`r2_score`	-inf to 1	Close to 1
Adj R2	Manual formula	-inf to 1	Close to 1

Complete Workflow (Copy-Paste Template):

```
# 1. Import
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.model_selection import train_test_split, cross_val_score,
GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np, joblib

# 2. Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# 3. Pipeline
pipe = Pipeline([('scaler', StandardScaler()), ('model', Ridge())])

# 4. GridSearch
grid = GridSearchCV(pipe, {'model__alpha': [0.01, 0.1, 1, 10]}, cv=5,
scoring='r2')
grid.fit(X_train, y_train)
```

```
# 5. Evaluate
y_pred = grid.predict(X_test)
print(f"R2: {r2_score(y_test, y_pred):.4f}")
print(f"RMSE: {np.sqrt(mean_squared_error(y_test, y_pred)):.2f}")

# 6. Save
joblib.dump(grid.best_estimator_, 'model.pkl')
```

8.6 Hum Kya Sikhe? — Course Summary

Linear Regression mein humne seekha:

1. **Introduction** — Supervised learning, continuous targets, LINE assumptions
2. **OLS** — Least squares, matrix form, beta calculation, R-squared
3. **Cost Functions** — MSE, MAE, RMSE, R2 — kab kya use karein
4. **Gradient Descent** — Mountain analogy, learning rate, Batch/SGD/Mini-batch
5. **Sklearn Implementation** — Complete pipeline, residual plots, joblib deployment
6. **Bias-Variance** — Underfitting/Overfitting, Ridge/Lasso/ElasticNet
7. **Scaling + CV** — StandardScaler, Pipeline, K-Fold, GridSearchCV
8. **Real World** — End-to-end project, coefficient interpretation, California Housing

Aage kya seekhenge?

- Decision Trees aur Random Forests
- Ensemble Methods (Bagging, Boosting)
- Neural Networks (Deep Learning)

Is course mein jo foundations build ki hain — features, targets, train-test split, metrics, overfitting, regularization, cross validation — yeh sab next topics mein bhi same rehti hain!